10. AI large model voice assistant (online/offline)

10. AI large model voice assistant (online/offline)
    1. Online/offline voice configuration
    2. Creating the auto-start service on boot (Systemd)
        2.1 Creating a startup script
        2.2 Creating the systemd service file
        2.3 Managing and debugging the service

1. Online/offline voice configuration

Note: You can choose the online LLM, ASR or TTS in this chapter, or refer to the corresponding chapter of <Online large model (Voice Interaction)>.

Before setting up the auto-start, first confirm that the voice assistant works normally in the mode you have chosen. The ASR, the LLM and the TTS can each choose online or offline; only the stages that are set to online need the network and the API credentials of that service provider.

1. **Locate the configuration file**:
   In your project code, find and open the configuration file:
   `~/yahboom_ws/src/largemodel/config/yahboom.yaml`
2. **Modify the configuration parameters**:
   Check the following parameters in the file and make sure that their values match what is shown below. If a parameter does not exist, add it.

```
asr:                                    # ASR node parameters / 语音节点参数
  ros__parameters:
    VAD_MODE: 2                         # VAD sensitivity / vad灵敏度
    sample_rate: 16000                  # ASR recording audio sample rate / asr录音音频采样率
    frame_duration_ms:  30              # VAD frame size in ms / vad帧大小单位ms
    use_oline_asr: False                # Whether to use online ASR (True for online, False for offline) / 是否使用在线asr识别（True使用在线，False使用离线）
    mic_serial_port: "/dev/ttyUSB0"     # Microphone serial port alias / 麦克风串口别名
    mic_index: 0                        # Microphone index / 麦克风索引
    language: 'en'                      # ASR language / asr语言
    regional_setting : "international"          #international：国际版  China：国内版 # international: International version, China: Domestic version

model_service:                          # Model service node parameters / 模型服务器节点参数
  ros__parameters:
    language: 'en'                      # Large model interface language / 大模型接口语言
    useolinetts: False                  # Whether to use online TTS (True for online, False for offline) / 是否使用在线语音合成（True使用在线，False使用离线）
```

```
    llm_platform: 'ollama'                # Currently selected large model platform / 当前选用的大模型平台
    # Available platforms: 'ollama', 'openrouter' / 可选平台: 'ollama',  'openrouter'
    regional_setting : "international"          #international：国际版  China：国内版 # international: International version, China: Domestic version
```

- Fully offline: `use_oline_asr: False`, `useolinetts: False`, `llm_platform: 'ollama'`.
- Fully online example: `use_oline_asr: True`, `useolinetts: True`, `llm_platform: 'tongyi'`. The online ASR/TTS still need the credentials of their respective service providers; follow articles 1 and 2 of this chapter.
- The ASR, the TTS and the LLM can be chosen in a mixed way; for example an online LLM with an offline ASR/TTS.

3. **Save the file** and **rebuild** the project to apply the changes:

```
cd ~/yahboom_ws
colcon build
source install/setup.bash
```

The online Tongyi LLM needs `qianwen_api_key` and the model name to be filled in; the online ASR needs `online_asr_api_key`; the online TTS needs the credentials of the corresponding service provider. In the offline mode, confirm that the Ollama model and the paths of the SenseVoiceSmall and the Piper models exist. See articles 1–3 of this chapter for the detailed fields and the examples.

The online mode additionally needs the network to be available at startup; the offline mode requires the local models to be installed.

> Note: if you are using the factory image, the scripts and the service related configuration are already complete, so the operations below are not needed

2. Creating the auto-start service on boot (Systemd)

Now we are going to create a `systemd` service so that `largemodel_control.launch.py` runs automatically when the system starts. In the online mode it is recommended to start after the network is ready; if the device connects slowly, you can check the connectivity of the interface and the service log after the service has started.

2.1 Creating a startup script

So that `systemd` can load the ROS2 environment correctly, the best practice is to create a simple `bash` script that wraps our startup command.

1. **Create the script file**:
   In the directory (`~/yahboom_ws/src/largemodel/`), create a file named `start_largemodel.sh`.

```
vim ~/yahboom_ws/src/largemodel/start_largemodel.sh
```

2. **Write the content of the script**:
   Copy and paste the following content into the script file.

```
#!/bin/bash

# Source the ROS2 jazzy environment
source /opt/ros/jazzy/setup.bash

# Source the Yahboom workspace environment
source /home/jetson/yahboom_ws/install/setup.bash

# Start the largemodel control script
ros2 launch largemodel largemodel_control.launch.py
```

Run `whoami` first to confirm the Linux user name and `echo $HOME` to confirm the home directory; replace `/home/jetson` in the script with the actual home directory of the device.

3. **Save and exit**
4. **Give the script the execute permission**:

```
chmod +x ~/yahboom_ws/src/largemodel/start_largemodel.sh
```

2.2 Creating the systemd service file

This is the most important step. We are telling the system that we now have a new service to manage.

1. **Create the service file**:
   You need `sudo` permission to create this file.

```
sudo vim /etc/systemd/system/largemodel.service
```

2. **Write the service configuration**:
   Copy and paste the following content in full into the service file.

```
[Unit]
Description=Robot Service
After=network-online.target sound.target graphical.target
Wants=network-online.target sound.target graphical.target

[Service]
Type=simple
User=jetson
Group=jetson
Environment=DISPLAY=:0
Environment=XDG_RUNTIME_DIR=/run/user/1000
Environment=PULSE_SERVER=unix:/run/user/1000/pulse/native
SupplementaryGroups=audio video
ExecStartPre=/bin/sleep 10
ExecStart=/home/jetson/yahboom_ws/src/largemodel/start_largemodel.sh
Restart=on-failure
StandardOutput=journal
StandardError=journal

[Install]
```

```
WantedBy=multi-user.target
```

Change the user name, the UID and the paths in `User`, `Group`, `XDG_RUNTIME_DIR`, `PULSE_SERVER` and `ExecStart` to the actual values of the device (you can look them up with `whoami`, `id -u` and `echo $HOME`). `ExecStart` has to point to the script created in the previous step, which needs the execute permission. The online mode uses `network-online.target`; the system also has to enable the wait-online service of the corresponding network manager in order to wait until the network is really ready.

3. **Save and exit**.

2.3 Managing and debugging the service

Now your service has been created, and we need to let `systemd` load it and set it to start on boot.

1. **Reload the `systemd` daemon** so that it reads the service file we just created:

```
sudo systemctl daemon-reload
```

2. **Set the service to start automatically on boot**:

```
sudo systemctl enable largemodel.service
```

3. **Start the service immediately**:

```
sudo systemctl start largemodel.service
```

4. **Check the status of the service**:
   This is the most important command for verifying whether the service is running successfully.

```
sudo systemctl status largemodel.service
```

   If you see `Active: active (running)`, the service process has started; if it is `failed`, keep reading the service log and check the configuration, the network and the audio devices.

5. **View the service log (essential for debugging)**:
   If the service fails to start, you can view all the real-time logs produced by the `ros2 launch` command with the following command, which is crucial for locating the problem.

```
journalctl -u largemodel.service -f
```

After the steps above, the system starts the `largemodel` voice service with the selected online or offline configuration.